

HiDiTingV100 音频算法插件

开发指南

文档版本 00B01

发布日期 2025-06-05

前言

概述

本文档主要介绍音频算法集成调试相关内容，包括使用方法和注意事项，用于指导客户更换使用第三方 SEA 算法。

产品版本

与本文档相对应的产品版本如下。

产品名称	产品版本
HiDiTing	V100

读者对象

本文档主要适用于以下工程师：

- 技术支持工程师
- 软件开发工程师

符号约定

在本文中可能出现下列标志，它们所代表的含义如下。

符号	说明
----	----

符号	说明
 危险	表示如不可避免则将会导致死亡或严重伤害的具有高等级风险的危害。
 警告	表示如不可避免则可能导致死亡或严重伤害的具有中等级风险的危害。
 注意	表示如不可避免则可能导致轻微或中度伤害的具有低等级风险的危害。
须知	用于传递设备或环境安全警示信息。如不可避免则可能会导致设备损坏、数据丢失、设备性能降低或其它不可预知的结果。 “须知”不涉及人身伤害。
 说明	对正文中重点信息的补充说明。 “说明”不是安全警示信息，不涉及人身、设备及环境伤害信息。

修改记录

文档版本	发布日期	修改说明
00B01	2025-06-05	第一次临时版本发布。

目 录

前言.....	i
1 概述.....	1
2 算法集成开发包.....	2
2.1 集成开发包说明.....	2
2.1.1 build 目录.....	2
2.1.2 include 目录.....	2
2.1.3 overlay 目录.....	3
2.1.4 component 目录.....	3
2.2 开发环境配置.....	3
2.2.1 工程路径配置.....	3
2.2.2 特性宏配置.....	4
2.2.3 Xtensa 编译工具链配置.....	4
2.3 增加或删除 C 库函数.....	4
2.4 Sample 插件介绍.....	5
2.4.1 插件目录结构.....	5
2.4.2 插件编译脚本配置.....	5
2.5 编译命令.....	6
3 创建算法工程.....	7
3.1 创建 SEA 插件工程.....	7
3.1.1 SEA 插件工程命名要求.....	7
3.1.2 新建 SEA 插件工程目录.....	7
3.1.3 新增 SEA 算法特性宏开关.....	8
3.1.4 SEA 插件编译脚本.....	8

3.2 创建 AEF 插件工程..... 8

4 开发算法插件 9

4.1 SEA 算法插件 9

4.1.1 SEA 插件接口说明 9

4.1.2 SEA 插件适配样例 13

4.1.3 SEA 插件注册方法 13

4.2 AEF 算法插件..... 13

5 加载和运行算法..... 14

5.1 烧录算法镜像包和 Sample 程序..... 14

5.2 启动 Sample 程序加载和运行算法 15

5.2.1 烧录 dsp_overlay.bin 到指定的 Flash 分区..... 15

5.2.2 执行 sample_sea 命令测试插件..... 15

插图目录

图 5-1 烧写完成示意图..... 15

Draft

表格目录

表 4-1 SEA 能力集合配置项 10

表 4-2 SEA 算法实例 PCM 属性 11

表 4-3 SEA 语音增强算法属性 12

表 4-4 SEA 语音 AI 算法属性 12

1 概述

本文档主要介绍音频算法集成相关内容，重点介绍了创建算法集成工程、开发算法插件、加载和运行算法等。可集成到 SAP（Smart Audio Platform）系统中的音频算法分别为 SEA 算法和 AEF 算法：

- SEA（Speech Enhancement and AI）算法是指语音增强及语音 AI 算法。
- AEF（Affect or Post process）算法是指音效算法或音频后处理算法。详情请参见《HiDiTingV100 AEF overlay 开发指南》。

2 算法集成开发包

2.1 集成开发包说明

2.2 开发环境配置

2.3 增加或删除 C 库函数

2.4 Sample 插件介绍

2.5 编译命令

2.1 集成开发包说明

集成开发包所在路径：工程根目录/application/brandy/audio/sample_dsp_overlay/sea

2.1.1 build 目录

build 目录：

- xtensa 子目录存放 DSP 相关的配置文件和编译脚本，建议只引用不修改。
 - hifi_rules.mak：配置编译规则、overlay 打包规则和依赖关系。
 - hifi_lsp.mak：配置 overlay 打包规则，被 hifi_rules.mak 调用。
 - xtensa_cfg.mak：配置 HiFi 工具链和 overlay 打包工具。
- xtensa/arch 目录：各个 overlay 的 memmap 配置。
- xtensa/tool 目录：overlay 打包工具。

2.1.2 include 目录

include 目录：

- 公共头文件，建议在插件源文件中包含。
- `td_type.h`: C 语言变量类型重定义
- `audio_alg.h`: 通用算法适配接口头文件。
- SEA 插件头文件，必须在插件源文件中包含。
- `audio_sea.h`: SEA 算法适配接口头文件。

2.1.3 overlay 目录

overlay 目录:

- component 头文件，必须在插件源文件中包含。
`component.h`: 通用算法插件注册接口，通过 `define_component` 接口将插件 entry 地址注册到 `component` 内存段中。
- `core_overlay` 模块作为插件统一入口和 C 库依赖，被编译到各个算法插件的 bin 文件中。
`core_overlay.c`: 重定向 C 库函数到相应的钩子函数，以及将 overlay 引导地址 `start` 指向可访问 `component` 内存的接口。

2.1.4 component 目录

component 目录:

- `sea/plugins` 子目录: SEA 算法插件工程目录。
 - `sea_lib0`: SEA 算法插件示例工程，开发者可参考该工程创建新的 SEA 算法插件工程。

2.2 开发环境配置

2.2.1 工程路径配置

须知

工程根目录中请不要出现空格。

`sap_idp.mak` 路径: 工程根目录

`/application/brandy/audio/sample_dsp_overlay/sea/sap_idp.mak`

修改 sap_idp.mak:

- IDP_OUT_DIR: 编译输出目录, 可修改到开发者自定义目录, 建议使用默认目录。

2.2.2 特性宏配置

sap_idp.mak 路径: 工程根目录

/application/brandy/audio/sample_dsp_overlay/sea/sap_idp.mak

修改 sap_idp.mak:

- CFG_SAP_SEA_LIB0_SUPPORT: 关闭该宏将不编译示例插件到 overlay 镜像中。
- SAP_SEA_LIB0_SUPPORT: “CFG_SAP_SEA_LIB0_SUPPORT” 打开时, 自动打开。
- 开发者开发新的 SEA 插件集成 SEA 算法, 应增加相应的宏开关, 宏命名格式如下:

```
CFG_SAP_SEA_XXX_SUPPORT=y
ifeq ($(CFG_SAP_SEA_XXX_SUPPORT), y)
    SAP_FEATURE_CONFIGS += -DSAP_SEA_XXX_SUPPORT
endif
```

2.2.3 Xtensa 编译工具链配置

修改 build/xtensa/xtensa_cfg.mak:

- XTENSA_HOME: 配置 HiFi 编译工具链安装路径。
- XTENSAD_LICENSE_FILE: 配置 HiFi 工具链 XTENSA License 文件。

2.3 增加或删除 C 库函数

说明

如果主镜像未注册相应的 C 库函数到该新增的 “overlay_func->func” 函数指针, 会导致 overlay 新增 C 库函数钩子。开发者可自行实现 C 库函数, 或是知会主镜像发布者发布新的镜像将相应的 C 库函数注册到 “over_func->func” 函数指针。

修改集成开发包的 overlay 目录下的 “core_overlay.h” 文件和 “core_overlay.c” 文件:

```
/* 在core_overlay.h的overlay_func结构体中, 新增一个C库同名函数类型定义 */
typedef struct {
    int (*func)(void);
```

```
} overlay_func;

/* 在core_overlay.c源码中，新增一个C库同名函数实现，重定向到主镜像的钩子上 */

int func(void)
{
    return g_overlay_func->func();
}
```

2.4 Sample 插件介绍

2.4.1 插件目录结构

component/sea/plugins/sea_lib0 工程目录结构：

- sea_lib0/include 目录：放置待集成 SEA 算法的头文件。
- sea_lib0/lib 目录：放置待集成 SEA 算法的依赖库文件。
- sea_lib0/src 目录：放置待集成 SEA 算法的源码文件。
- sea_lib0/xsea_lib0_comp.c 源文件：将待集成的 SEA 算法集成到 SEA 插件中。

2.4.2 插件编译脚本配置

- 在“sap_idp.mak”配置文件中，定义相应的宏开关：

```
CFG_SAP_SEA_LIB0_SUPPORT=y
ifeq ($(CFG_SAP_SEA_LIB0_SUPPORT), y)
    SAP_FEATURE_CONFIGS += -DSAP_SEA_LIB0_SUPPORT
endif
```

- 在 component/sea/plugins/Makefile 编译脚本中，增加新增的 SEA 插件目录：

```
=====
===
sea_libx_$(CFG_SAP_SEA_LIB0_SUPPORT) += sea_lib0
=====
===
```

- component/sea/plugins/sea_lib0/Makefile 编译脚本内容如下：

```
CUR_DIR := $(shell if [ "$$PWD" != "" ]; then echo $$PWD; else pwd; fi)
# 从当前目录回退到集成开发包所在路径
SAP_CORE_DIR := $(realpath $(CUR_DIR)/../../..)
# 包含集成开发包编译环境配置和编译工具链配置文件
include $(SAP_CORE_DIR)/sap_idp.mak
```

```
include $(XTENSA_BUILD_DIR)/xtensa_cfg.mak

# 编译待集成的SEA算法源码成静态库 lib_sea_lib0.a

ifdef BUILD_LIB
TARGET := libsea_lib0_$(CFG_SAP_DSP_ARCH)
CFLAGS_EXT :=
INCLUDES := -linclude -I$(SAP_CORE_DIR)/include
LIB_INSTALL_DIR := lib

/* 只编译待集成SEA算法的源文件 */

SRCFILE := $(wildcard src/*.c)

# 编译待集成的SEA算法插件，并链接算法静态库lib_sea_lib0.a 到 sea_lib0.bin中
else
# TARGET命名规则：当前只有4个名字(sea_lib0/ sea_lib1/ sea_lib2/ sea_lib3) 可用，
# 分别对应soc_uapi_sea.h接口中的UAPI_SEA_LIB_EXT0~3。

TARGET := sea_lib0
CFLAGS_EXT :=
INCLUDES := -linclude -I$(SAP_CORE_DIR)/include

# 链接待集成SEA算法的静态库 lib_sea_libx.a，如还依赖其他静态库也链接进来。
LIBS := -l:lib/libsea_lib0_$(CFG_SAP_DSP_ARCH).a

# 只编译待集成SEA算法的插件适配源文件
SRCFILE := xasea_lib0_comp.c
endif

# 包含编译规则和打包规则配置文件
include ../rules.mak
```

2.5 编译命令

在算法集成开发包所在路径下，执行以下命令：

步骤 1 清除编译结果：make clean

步骤 2 编译待集成算法静态库：make lib

步骤 3 编译算法插件 bin 文件：make all

----结束

3 创建算法工程

3.1 创建 SEA 插件工程

3.2 创建 AEF 插件工程

3.1 创建 SEA 插件工程

3.1.1 SEA 插件工程命名要求

- SEA 算法插件镜像 bin 文件命名要求：
当前只有 4 个名字可用：sea_lib0、sea_lib1、sea_lib2、sea_lib3。对应“soc_uapi_sea.h”中定义的 SEA LIB ID：UAPI_SEA_LIB_EXT0、UAPI_SEA_LIB_EXT1、UAPI_SEA_LIB_EXT2、UAPI_SEA_LIB_EXT3。若使用其他命名会导致插件无法通过 uapi_sea_load_engine 接口加载。
- SEA 插件工程命名建议与插件镜像 bin 文件名一致。

3.1.2 新建 SEA 插件工程目录

新建工程 sea_lib1 到 component/sea/plugins 目录下，工程目录结构如下：

- sea_lib1/include 目录：放置待集成 SEA 算法的头文件。
- sea_lib1/lib 目录：放置待集成 SEA 算法的依赖库文件。
- sea_lib1/src 目录：放置待集成 SEA 算法的源码文件。
- sea_lib1/xasea_lib1_comp.c 源文件：将待集成的 SEA 算法集成到 SEA 插件中。

3.1.3 新增 SEA 算法特性宏开关

在 “sap_idp.mak” 配置文件中，定义相应的宏开关：

```
CFG_SAP_SEA_LIB1_SUPPORT=y
ifeq ($(CFG_SAP_SEA_LIB1_SUPPORT), y)
    SAP_FEATURE_CONFIGS += -DSAP_SEA_LIB1_SUPPORT
endif
```

3.1.4 SEA 插件编译脚本

- 在 component/sea/plugins/Make 编译脚本中，增加新增的 SEA 插件目录：

```
sea_libx_$(CFG_SAP_SEA_LIB1_SUPPORT) += sea_lib1
```

- component/sea/plugins/sea_lib1/Makefile 编译脚本内容如下：

```
CUR_DIR := $(shell if [ "$$PWD" != "" ]; then echo $$PWD; else pwd; fi)
SAP_CORE_DIR := $(realpath $(CUR_DIR)/../..)
include $(SAP_CORE_DIR)/sap_idp.mak
include $(XTENSA_BUILD_DIR)/xtensa_cfg.mak
ifdef BUILD_LIB
    TARGET := libsea_lib1_$(CFG_SAP_DSP_ARCH)
    CFLAGS_EXT :=
    INCLUDES := -linclude -I$(SAP_CORE_DIR)/include
    LIB_INSTALL_DIR := lib
    SRCFILE := $(wildcard src/*.c)
    TARGET := sea_lib1
    CFLAGS_EXT :=
    INCLUDES := -linclude -I$(SAP_CORE_DIR)/include
    LIBS := -l:lib/libsea_lib1_$(CFG_SAP_DSP_ARCH).a
    SRCFILE := xasea_lib1_comp.c
endif
include ../rules.mak
```

3.2 创建 AEF 插件工程

详细内容请参见《HiDiTingV100 AEF overlay 开发指南》。

4 开发算法插件

4.1 SEA 算法插件

4.2 AEF 算法插件

4.1 SEA 算法插件

4.1.1 SEA 插件接口说明

SEA 插件接口说明如下：

- audio_sea.h 定义结构体 aal_sea_entry 成员变量（即 SEA 插件属性）：
 - name：字符串，算法库名称。
 - sea_id：SEA 插件 ID，参考 sea_struct.h 中定义的枚举类型 sea_lib_id（注意：开发者自定义算法只能使用 4 个 ID，即 SEA_LIB_ID_EXT0 ~ SEA_LIB_ID_EXT3）。
 - version：算法版本号，详细定义参考 audio_lib_version。
 - description：字符串，描述算法简介及开发者名称等。
- audio_sea.h 定义结构体 aal_sea_entry 成员接口（即 SEA 插件适配接口）：
 - get_eng_caps：输出算法能力集合，至少输出一个能力配置。（必备接口）
 - get_def_pcm_attr：输出算法的默认 PCM 属性。（必备接口）
 - create：创建算法实例并输出唯一的实例句柄给 SEA 模块。（必备接口）
 - destroy：销毁实例句柄对应的算法实例，并释放相关资源。（必备接口）
 - proc_frame：执行实例句柄对应的算法实例来处理输入的 PCM 数据，通过 reg_data_output 接口注册的数据输出回调接口将处理后的数据输出给调用者，

通过 reg_event_report 接口注册的事件上报回调接口将处理后的结果上报给调用者。(必备接口)

- set_enable: 使能或停止实例句柄对应的算法实例。(可选接口)
 - set_scr_buf: 设置系统分配给算法实例使用的 Scratch Buffer (多实例共用的临时缓存, 仅在算法实例的运算周期内有效, 不可用于缓存运算周期之外的数据)。(必备接口)
 - get_eng_attr: 获取实例句柄对应的算法实例的属性, 未实现时可以直接返回不支持。(可选接口)
 - set_eng_attr: 设置实例句柄对应的算法实例的属性, 未实现时可以直接返回不支持。(可选接口)
 - set_input_attr: 设置实例句柄对应的算法实例处理前的 PCM 数据格式, 至少支持一种格式设置。(可选接口)
 - set_output_attr: 设置实例句柄对应的算法实例处理后的 PCM 数据格式, 至少支持一种格式设置。(可选接口)
 - get_item_sets: 获取实例句柄对应的算法实例所支持的条目列表 (如命令词列表或 VoiceID 列表), 命令词算法或声纹识别算法必选, 其他算法可选。(可选接口)
 - reg_data_output: 注册数据输出回调函数指针给实例句柄对应的算法实例。(必备接口)
 - reg_event_report: 注册事件上报回调函数指针给实例句柄对应的算法实例。(必备接口)
- SEA 算法能力集合配置项如表 4-1 所示 (最大可配置 1 个语音增强选项和 3 个语音 AI 选项)。

表4-1 SEA 能力集合配置项

配置项	默认值	说明
sea_eng_sel. eng_ind [SEA_ECH_A FE_IND].type	无	参考 sea_struct.h 中定义的枚举类型 sea_afe_type 及其注释。 必须匹配算法特性。
sea_eng_sel. eng_ind [SEA_ECH_A FE_IND].type. lib_id	无	参考 sea_struct.h 中定义的枚举类型 sea_lib_id 及其注释。 开发者自定义算法只能使用 SEA_LIB_ID_EXT0 ~ SEA_LIB_ID_EXT3。

配置项	默认值	说明
sea_eng_sel. eng_ind [SEA_ECH_A AI_IND].type	无	参考 sea_struct.h 中定义的枚举类型 sea_aai_type 及其注释。 必须匹配算法特性。
sea_eng_sel. eng_ind [SEA_ECH_A AI_IND].type.li b_id	无	参考 sea_struct.h 中定义的枚举类型 sea_lib_id 及其注释。 开发者自定义算法只能使用 SEA_LIB_ID_EXT0 ~ SEA_LIB_ID_EXT3。

说明

语音算法最大可配置 1 个语音增强选项和 3 个语音 AI 选项。大多数算法只支持一个配置选项，但也有语音全栈算法要求配置组合选项的，比如语音增强 + 声纹识别 + 唤醒词 + 命令词组合。

- SEA 算法实例属性主要如表 4-2 所示。

表4-2 SEA 算法实例 PCM 属性

属性项	默认值	说明
sea_pcm_attr. buf_dur_ms	无	输入的 PCM 缓存时长，单位：ms。（可不处理）
sea_pcm_attr. in_pcm	无	MIC 输入的 PCM 属性：采样率、位宽、通道数及每帧采样点数。
sea_pcm_attr. ref_pcm	无	参考信号的 PCM 属性：采样率、位宽、通道数及每帧采样点数。
sea_pcm_attr. out_pcm[]	无	输出处理后的语音信号 PCM 属性：采样率、位宽、通道数及每帧采样点数。某些算法可能输出多路数据，开发者按需适配即可。

- SEA 语音增强算法属性如表 4-3 所示。（高阶适配才需要，必须包含“audio_sea_struct.h”头文件）

表4-3 SEA 语音增强算法属性

属性项	默认值	说明
sea_afe_attr.type	无	语音增强算法类型：语音增强、通话增强、人声增强等。 开发者应根据算法类别配置相应的选项。
sea_afe_attr.mode	无	工作模式：正常模式、低功耗模式、高性能模式。
sea_afe_attr.u.see	无	语音增强属性，提供开关增强算法特性的选项。
sea_afe_attr.u.vqe	无	通话增强属性，提供开关通话算法特性的选项。
sea_afe_attr.u.ahe	无	通话人声属性，提供开关通话算法特性的选项。

- SEA 语音 AI 算法属性如表 4-4 所示。

表4-4 SEA 语音 AI 算法属性

属性项	默认值	说明
sea_aai_attr.type	无	语音 AI 算法类型：唤醒词、语音识别、声纹识别、场景识别等。 开发者应根据算法类别配置相应的选项。
sea_aai_attr.mode	无	工作模式：正常模式、低功耗模式、高性能模式。
sea_aai_attr.u.kws	无	语音唤醒属性：灵敏度、语音最大时长、VAD 端点门限。
sea_aai_attr.u.asr	无	语音识别属性：NLP 使能、语音最大时长、VAD 端点门限。
sea_aai_attr.u.vid	无	声纹识别属性：灵敏度、语音最小时长、最大识别数。
sea_aai_attr.u.asd	无	场景识别属性：时延，单位：ms。

4.1.2 SEA 插件适配样例

参考算法集成开发包路径下的适配样例：

工程根目录

/application/brandy/audio/sample_dsp_overlay/sea/component/sea/plugins/sea_lib0/
xasea_lib0_comp.c

4.1.3 SEA 插件注册方法

SEA 插件注册参考样例及说明如下。

```
/* 定义一个全局的插件 entry结构体 */
static ha_sea_entry ha_sea_lib_entry = {
    .name = (const td_char *)"sea_lib0",

    /* 开发者自定义插件只能使用SEA_LIB_ID_EXT0 ~ SEA_LIB_ID_EXT3 */

    .sea_id = SEA_LIB_ID_EXT0,
    .version.u32 = 0x10000001,
    .description = (const td_char *)"Sea lib0 developed by XXX",
    .create = sea_lib_create,
    .destroy = sea_lib_destroy,
    .proc_frame = sea_lib_proc_frame,
    .set_enable = sea_lib_set_enable,
    .set_scr_buf = sea_lib_set_scr_buf,
    .get_item_sets = TD_NULL,
    .get_eng_caps = sea_lib_get_eng_caps,
    .get_eng_attr = TD_NULL,
    .set_eng_attr = TD_NULL,
    .set_input_attr = TD_NULL,
    .set_output_attr = TD_NULL,
    .get_def_pcm_attr = sea_lib_get_def_pcm_attr,
    .reg_data_output = sea_lib_reg_data_output,
    .reg_event_report = sea_lib_reg_event_report
};

/* 将插件 entry地址注册到.component 地址段中 */
define_component(SEA, ha_sea_lib_entry);
```

4.2 AEF 算法插件

详细内容请参见《HiDiTingV100 AEF overlay 开发指南》。

5 加载和运行算法

5.1 烧录算法镜像包和 Sample 程序

5.2 启动 Sample 程序加载和运行算法

5.1 烧录算法镜像包和 Sample 程序

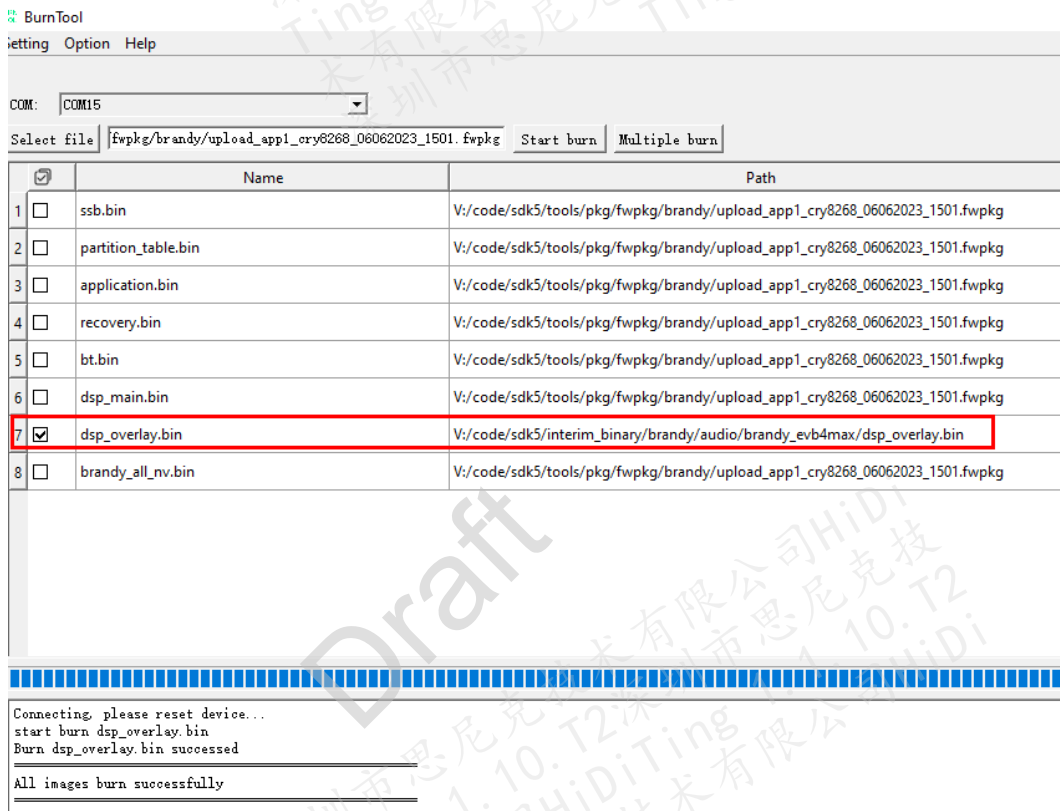
加载 dsp_overlay.bin 到单板，当前 SEA 插件镜像 bin 文件统一打包到“dsp_overlay.bin”文件中。开发者编译生成的“dsp_overlay.bin”的路径：out/bin/dsp_overlay.bin。

待 Flash 分区规划完后，SEA 插件镜像 bin 文件将会单独打包。

5.2 启动 Sample 程序加载和运行算法

5.2.1 烧录 dsp_overlay.bin 到指定的 Flash 分区

图5-1 烧写完成示意图



- 步骤 1 启动 BurnTool 工具，完成一次整机镜像烧写。具体烧写步骤请参见《HiDiTingV100 BurnTool 工具 使用指南》。
- 步骤 2 点击 “dsp_overlay.bin” 对应的 Path 文本框，选择 “5.1 烧录算法镜像包和 Sample 程序” 中打包生成的 “dsp_overlay.bin” 文件的路径
application/brandy/audio/sample_lsp/out/bin/dsp_overlay.bin。
- 步骤 3 点击 “Start burn” 启动烧写 dsp_overlay.bin。

----结束

5.2.2 执行 sample_sea 命令测试插件

- 步骤 1 修改 sample_sea.c 文件，以适配 AT 命令。

1. 对 sample_sea_parse_lib_type 函数进行修改, 修改如下:

```
static td_s32 sample_sea_parse_lib_type(td_char *param, sample_sea_context *ctx)
{
    #if 0
        if (strcmp(param, "kws") == 0) {
            ctx->sea_inst.lib_id = UAPI_SEA_LIB_KWS;
            return EXT_SUCCESS;
        }
    #endif
    if (strcmp(param, "ext0") == 0) {
        ctx->sea_inst.lib_id = UAPI_SEA_LIB_EXT0;
        return EXT_SUCCESS;
    }
    if (strcmp(param, "ext1") == 0) {
        ctx->sea_inst.lib_id = UAPI_SEA_LIB_EXT1;
        return EXT_SUCCESS;
    }
    return EXT_FAILURE;
}
```

2. 对 init_sea_eng 函数进行修改, 修改如下:

```
static td_void init_sea_eng(uapi_sea_eng_sel *sea_eng, const sample_sea_inst *sea_inst)
{
    (td_void)memset_s(sea_eng, sizeof(*sea_eng), 0, sizeof(*sea_eng));
    sea_eng->afe.type = UAPI_SEA_AFE_SEE;
    sea_eng->afe.lib_id = sea_inst->lib_id;
    #if 0
        sea_eng->afe.lib_id = UAPI_SEA_LIB_SEE;
        sea_eng->aai[0].type = UAPI_SEA_AAI_KWS;
        sea_eng->aai[0].lib_id = sea_inst->lib_id;
    #endif
}
```

3. 对 sample_sea_open_sea 函数进行修改, 修改如下:

```
static td_s32 sample_sea_open_sea(sample_sea_context *ctx)
{
    td_s32 ret;
    uapi_sea_eng_sel sea_eng;
    uapi_sea_attr sea_attr;
    sample_sea_inst *sea_inst = &ctx->sea_inst;
    init_sea_eng(&sea_eng, sea_inst);
    #if 0
        ret = uapi_sea_load_engine(UAPI_SEA_LIB_SEE, "imedia_2mic");
    #endif
}
```

```

    if (ret != EXT_SUCCESS) {
        sap_err_log_fun(uapi_sea_load_engine, ret);
    }
    ret = uapi_sea_load_engine(sea_inst->lib_id, "imedia_keyword");
    if (ret != EXT_SUCCESS) {
        sap_err_log_fun(uapi_sea_load_engine, ret);
    }
#endif

    ret = uapi_sea_load_engine(sea_inst->lib_id, "sea_lib");
    if (ret != EXT_SUCCESS) {
        sap_err_log_fun(uapi_sea_load_engine, ret);
    }
    ret = uapi_sea_get_default_attr(&sea_eng, &sea_attr);
    if (ret != EXT_SUCCESS) {
        sap_err_log_fun(uapi_sea_get_default_attr, ret);
        goto out0;
    }
    sea_attr.ref_pcm.channels = SEA_AI_REF_CH_NUM;
    ret = uapi_sea_create(&sea_inst->h_sea, &sea_eng, &sea_attr);
    if (ret != EXT_SUCCESS) {
        sap_err_log_fun(uapi_sea_create, ret);
        goto out0;
    }
    /* Register SEA event process callback */
    ret = uapi_sea_register_event_proc(sea_inst->h_sea, sample_sea_event_proc, ctx);
    if (ret != EXT_SUCCESS) {
        sap_err_log_fun(uapi_sea_register_event_proc, ret);
        goto out1;
    }
    (td_void)memset_s(&sea_inst->phs_sets, sizeof(sea_inst->phs_sets), 0, sizeof(sea_inst->phs_sets));
#endif

    sea_inst->item_sets.size = SEA_PHRASE_COUNT;
    sea_inst->item_sets.array = sea_inst->phs_sets.phrase;
    ret = uapi_sea_get_phrase_sets(sea_inst->h_sea, &sea_inst->item_sets);
    if (ret != EXT_SUCCESS) {
        sap_err_log_fun(uapi_sea_get_phrase_sets, ret);
        goto out1;
    }
    sea_inst->phs_sets.phs_count = sea_inst->item_sets.count;
#endif

    return EXT_SUCCESS;
out1:

```



```

        (td_void)uapi_sea_destroy(sea_inst->h_sea);
out0:
#ifdef 0
        (td_void)uapi_sea_unload_engine(sea_inst->lib_id, "imedia_keyword");
        (td_void)uapi_sea_unload_engine(UAPI_SEA_LIB_SEE, "imedia_2mic");
#endif
        (td_void)uapi_sea_unload_engine(sea_inst->lib_id, "sea_lib");
        return ret;
}

```

4. 对 sample_sea_close_sea 函数进行修改, 修改如下:

```

static td_void sample_sea_close_sea(sample_sea_context *ctx)
{
    td_s32 ret;
    ret = uapi_sea_destroy(ctx->sea_inst.h_sea);
    if (ret != EXT_SUCCESS) {
        sap_err_log_fun(uapi_sea_destroy, ret);
    }
#ifdef 0
    ret = uapi_sea_unload_engine(ctx->sea_inst.lib_id, "imedia_keyword");
    if (ret != EXT_SUCCESS) {
        sap_err_log_fun(uapi_sea_unload_engine, ret);
    }
    ret = uapi_sea_unload_engine(UAPI_SEA_LIB_SEE, "imedia_2mic");
    if (ret != EXT_SUCCESS) {
        sap_err_log_fun(uapi_sea_unload_engine, ret);
    }
#endif
    ret = uapi_sea_unload_engine(ctx->sea_inst.lib_id, "sea_lib");
    if (ret != EXT_SUCCESS) {
        sap_err_log_fun(uapi_sea_unload_engine, ret);
    }
}

```

步骤 2 请参考《HiDiTingV100 软件开发指南》的“启动编译-makefile 编译”章节重新编译“application.bin”并烧录到指定的 Flash 分区。

步骤 3 执行“sample_sea”命令测试插件, 用于插件测试的 AT 命令使用方法如下:

```

examples:
AT^audio=sample_sea 10 null ext0 pdm0
AT^audio=sample_sea 10 null ext1 pdm0

```

步骤 4 测试结果: 执行未报错并返回<sample execute succeeded.>。

----结束

Draft